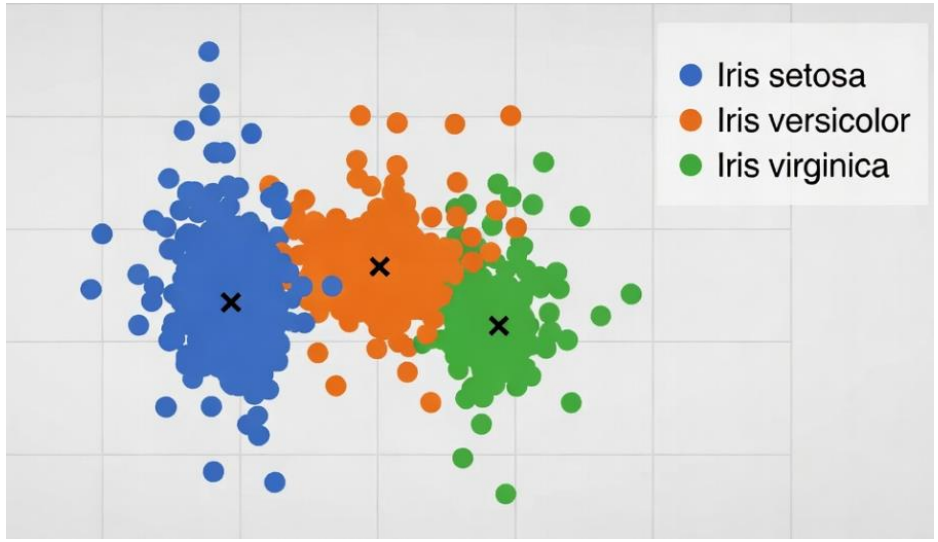
K-means Clustering

- Unsupervised Learning -



SIT@KMUTT

2026

What K-means clustering is …

- K-means clustering is one of the most popular and widely used unsupervised machine learning algorithms. It is used to group similar data points together into clusters, without prior knowledge of the group labels.
- The "K" in K-means is the number of clusters you want to find (you choose K in advance).
- Sensitive to initial centroid placement (can converge to poor solutions).
- You must specify K in advance (use methods like the Elbow Method or Silhouette Score to help choose your best K)
- Not suitable for non-numerical (categorical) data. Even if you encode categories numerically (like one-hot encoding), the Euclidean distance may become misleading.
- Instead, use K-modes clustering which is designed specifically for categorical data. Measure dissimilarity by number of mismatches (like Hamming distance).
- Or use K-prototypes clustering which combines K-Means (for numerical) and K-Modes (for categorical).

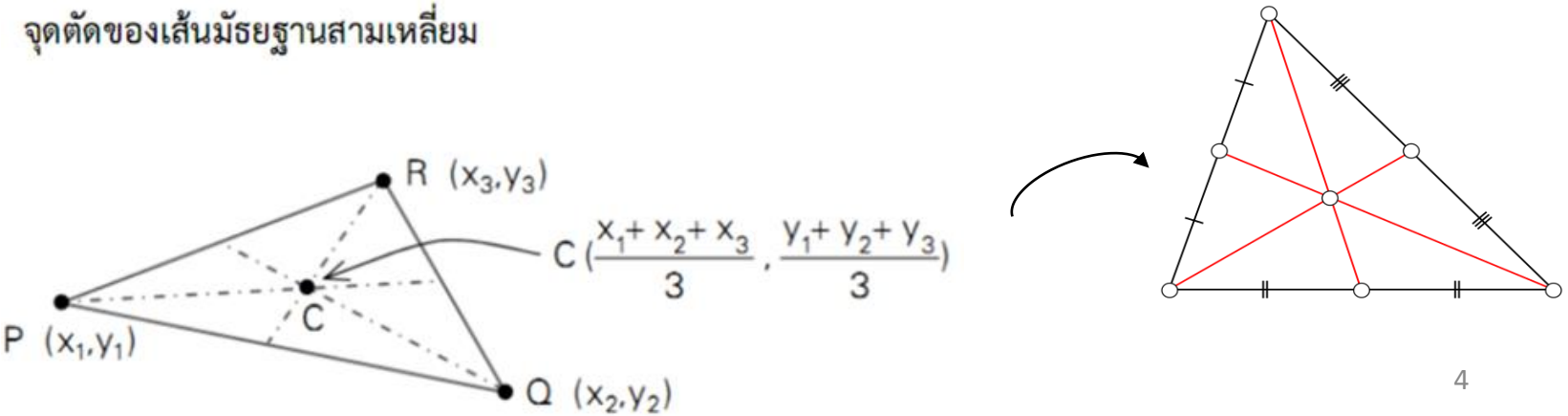
Overview: K-means, data variables, and limitations

- The basic k-means clustering is based on a non-deterministic algorithm. This means that running the algorithm several times on the same data, could give different results.
- The non-deterministic nature of k-means is due to its random selection of data points as initial centroids.
- A simple way to cluster nominal/categorical data is by using k-modes algorithm, which is similar to k-means in principle but uses modes instead of means. It defines clusters based on the number of matching categories between data points.

K-means clustering

- K-means is an *unsupervised* machine learning algorithm used to find groups of observations (clusters) that share similar characteristics.
- *K*-means divides the data into k clusters by minimizing the *sum of the squared distances* of each record/data point to the *mean* of its assigned cluster.
- This is referred to as the *within-cluster sum of squares* or *within-cluster SS (WCSS)*.
- *K*-means does not ensure the clusters will have the same size, but finds the clusters that are the best separated

Example of computing a centroid of a triangle



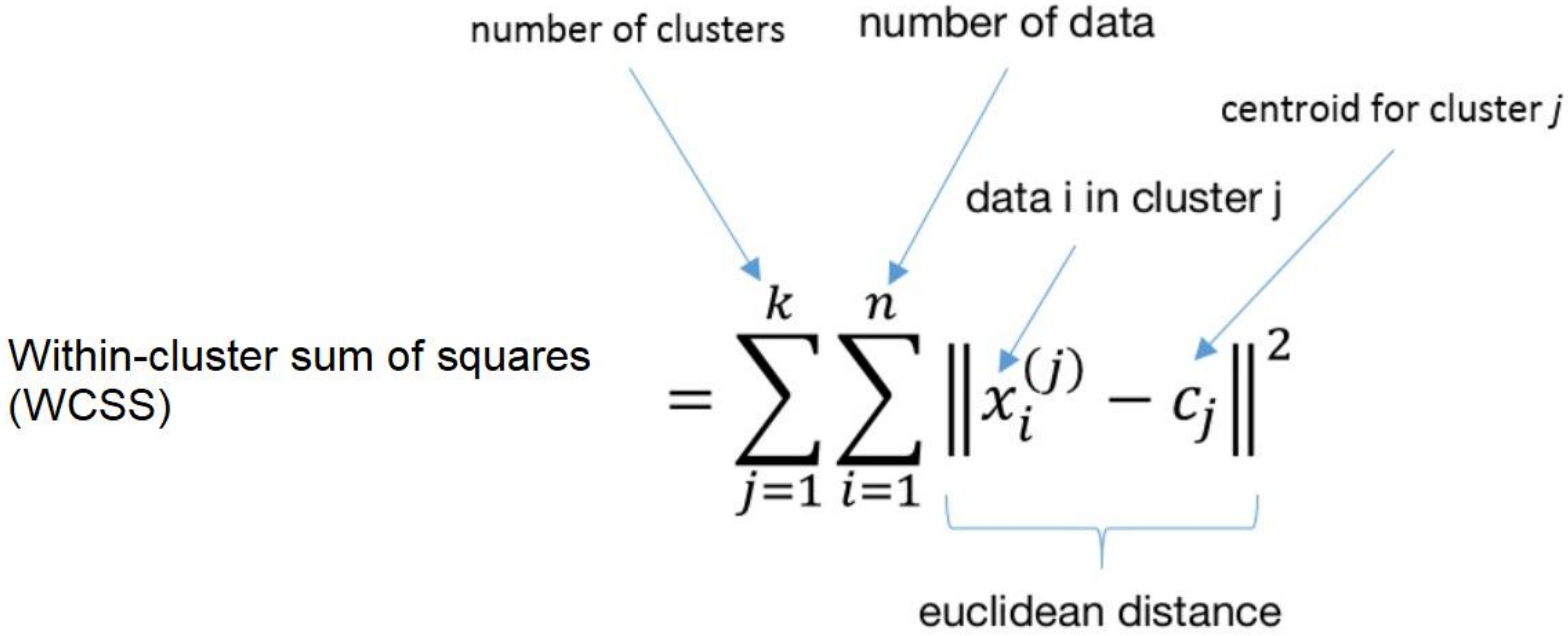
K-means clustering algorithm

The most common partitioning method is the K-means cluster analysis. Conceptually, the k-means algorithm is as follows.

1. Select *k* centroids (*k* rows of the dataset are chosen at random)
2. Assign each data point to its closest centroid
3. Recalculate the centroids as the average of all data points in a cluster
4. Reassign data points to their closest centroids again
5. Repeat the steps no. 3 and 4 until the observations (data points) are not reassigned or the maximum number of iterations is reached

K-means clustering algorithm

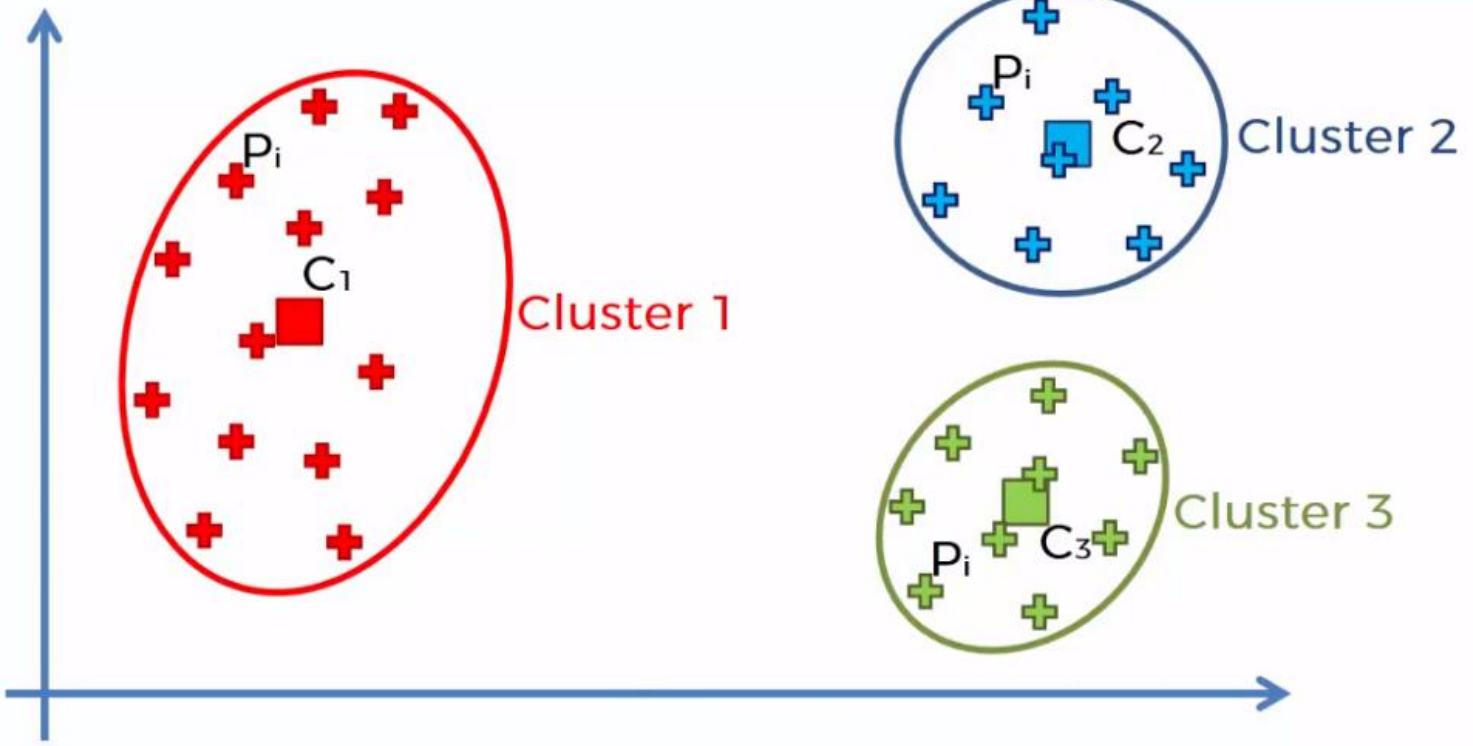
R uses an efficient algorithm by Hartigan and Wong (1979) that partitions the observations into ***k*** groups such that the within-cluster **sum of squares (WCSS)** of the observations to their assigned cluster centers (centroid) is a minimum. This means that in the steps no. 2 and 4, each observation is assigned to the cluster with the smallest value of *WCSS.*



Sources:

1. https://www.r-statistics.com/2013/08/k-means-clustering-from-r-in-action/
2. https://www.slideshare.net/radiohead0401/cluster-analysis-assignment-update by Billy Yang

Illustration of the simple 3-means clusters



$$\text{WCSS} = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster 2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster 3}} \text{distance}(P_i, C_3)^2$$

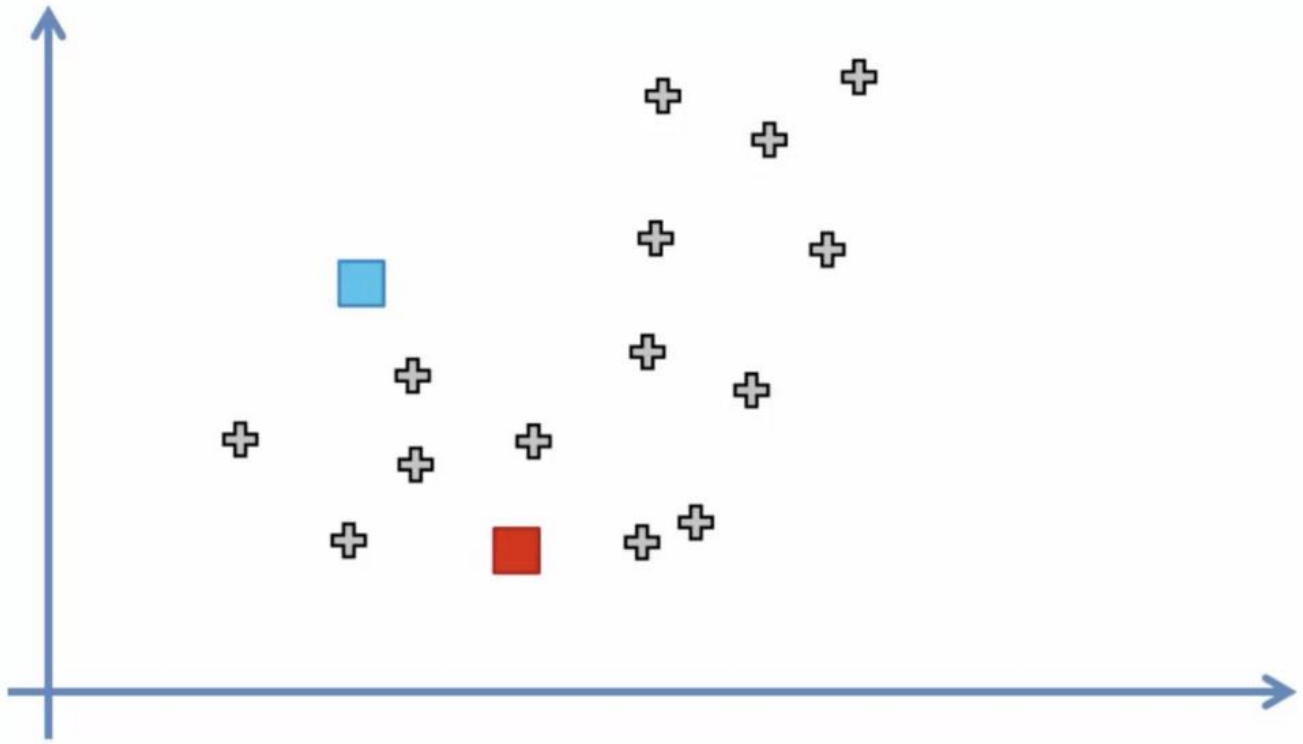
Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

Two different procedures in assigning a point to clusters

1. A sample can be allocated to just one cluster (**hard clustering**)
   - E.g., k-means
2. Or the sample can be distributed among several clusters (**fuzzy clustering**), also referred to as **soft clustering** or **soft *k*-means**
   - Fuzzy clustering allows that one sample belongs, to a certain degree, to several groups, according some assigned weighted value
   - E.g., an apple can be red or green (hard clustering), but an apple can also be red AND green (fuzzy clustering). Here, the apple can be red to a certain degree as well as green to a certain degree
   - E.g., **fuzzy C-means clustering (FCM)** algorithm

Credit: https://en.wikipedia.org/wiki/Fuzzy_clustering

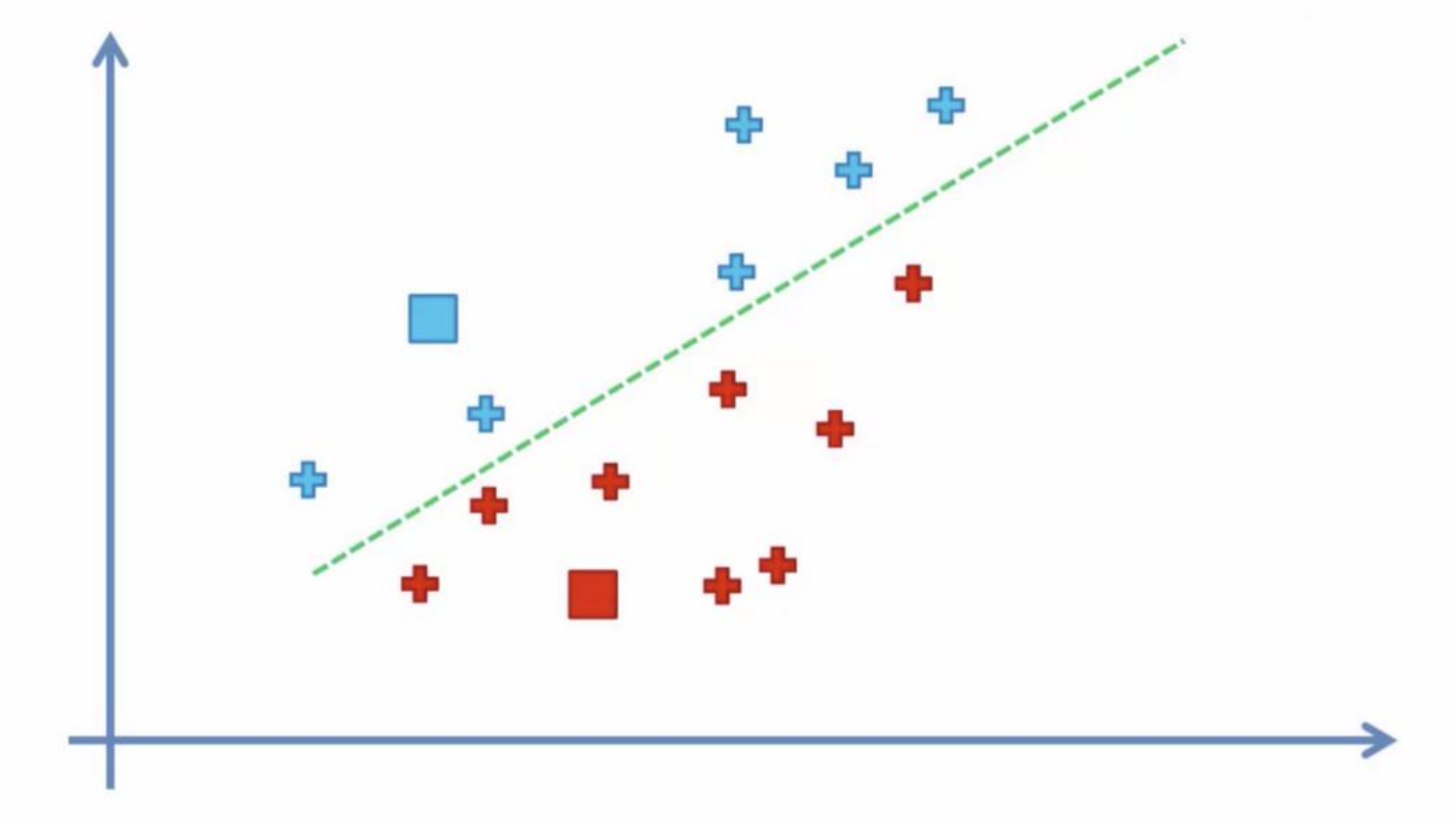
Simple K-means illustration



Assume that we randomly choose two points (here, red and green ones) for being centroids, which are the centers of corresponding clusters

Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

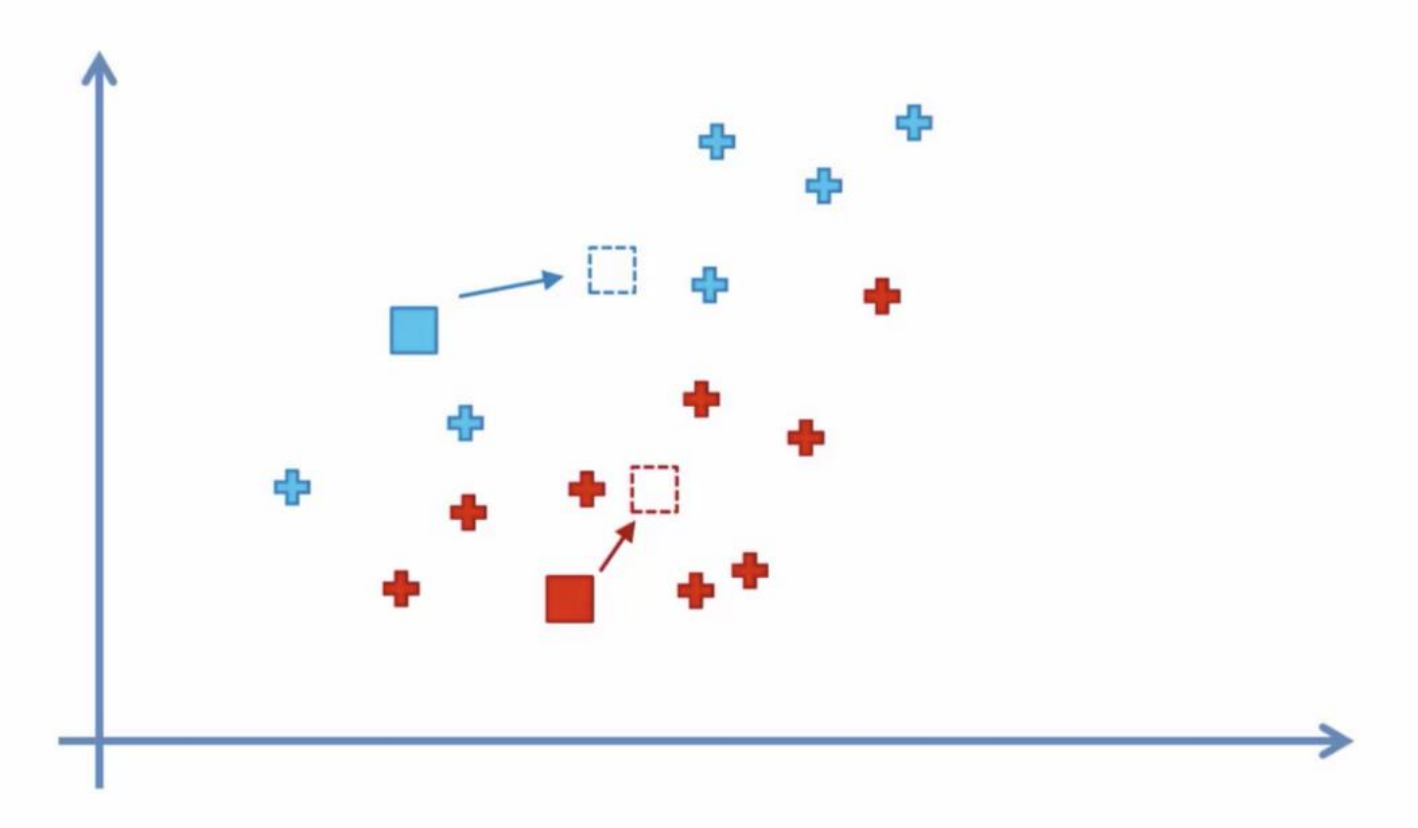
Simple k-means illustration



Assign each point to the closest cluster’s centroid

Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

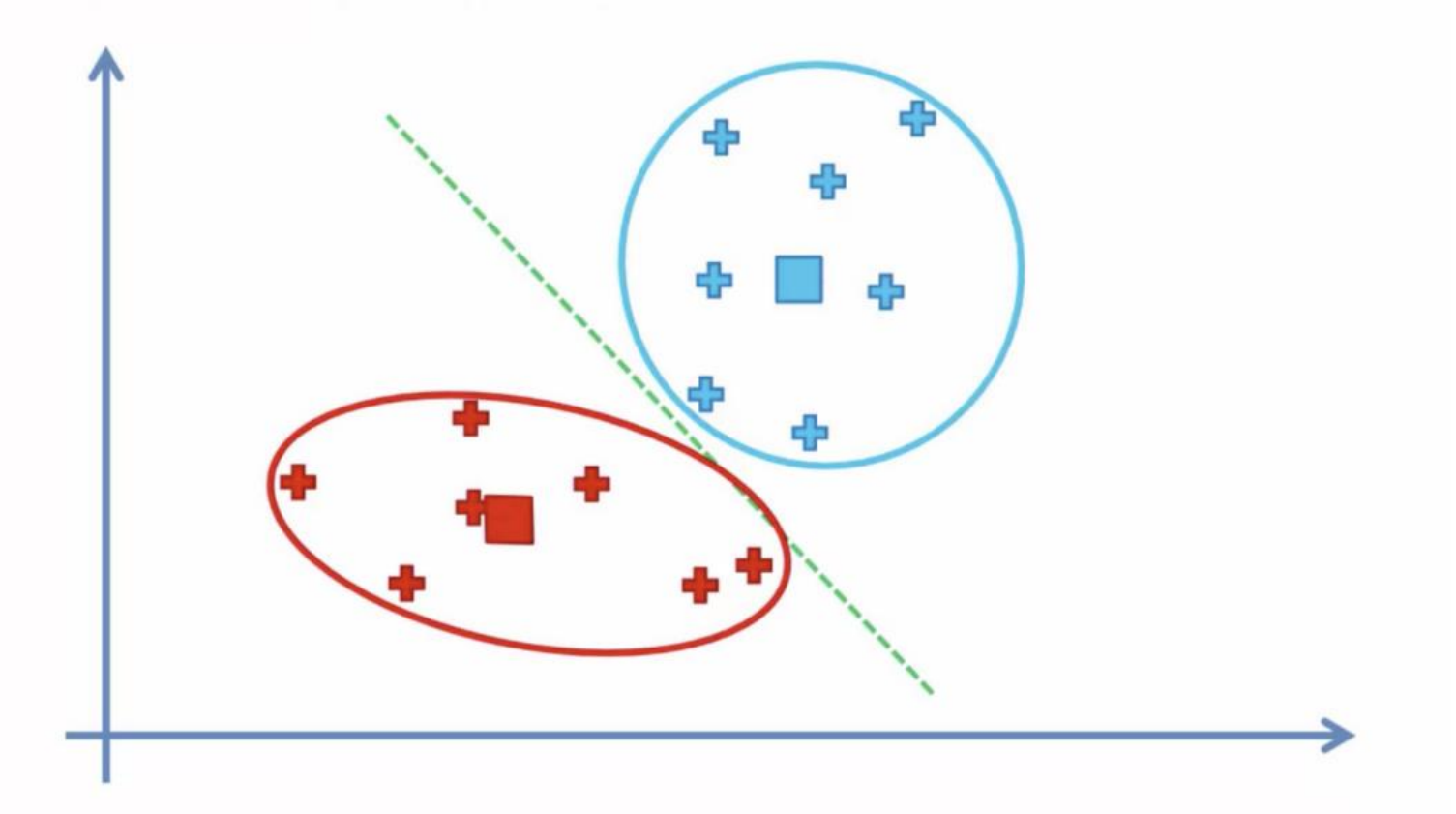
Simple k-means illustration



Recalculate the new cluster’s centroid

Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

Simple k-means illustration

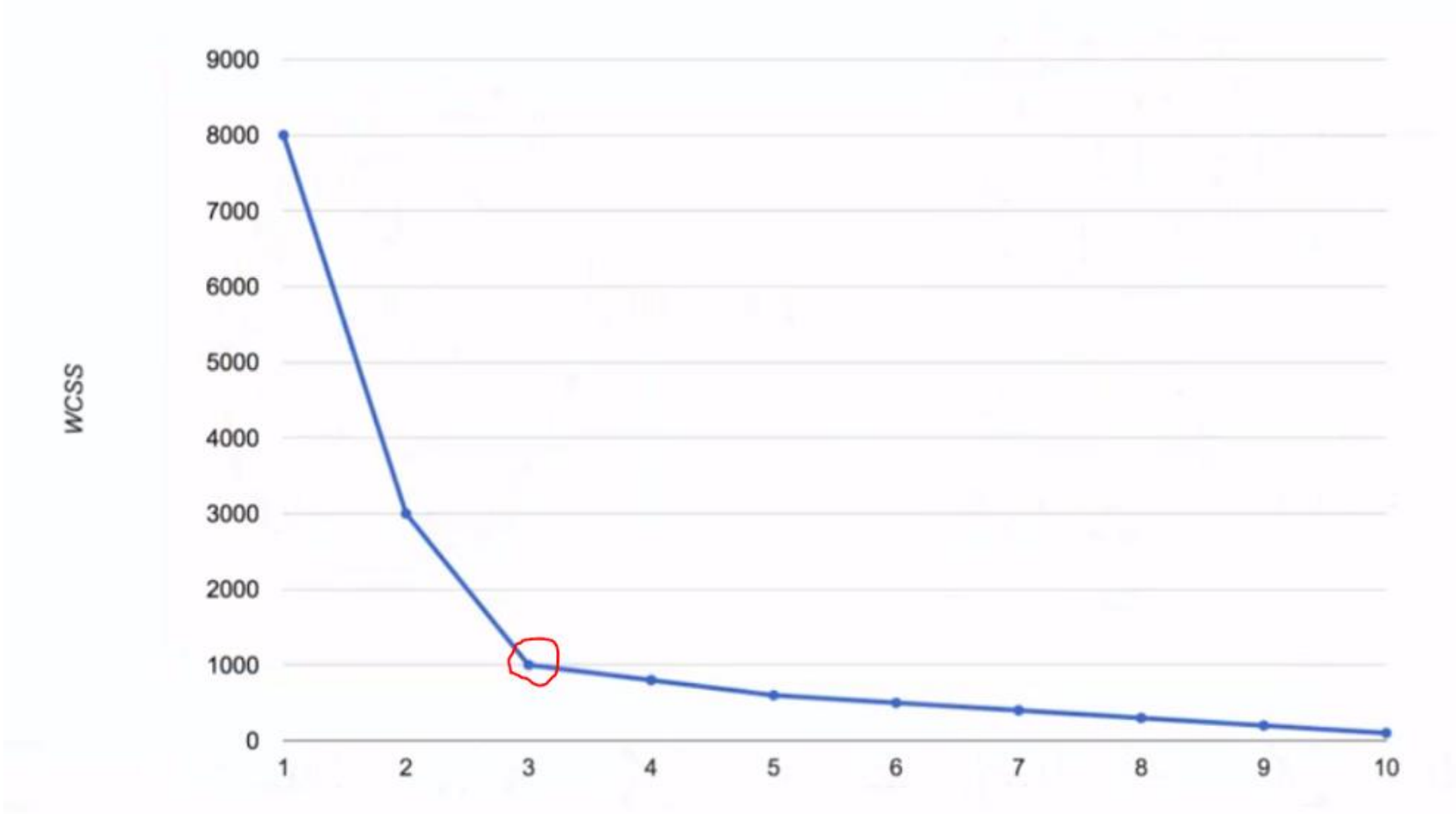


OK now.

Repeat the previous two steps until no more assignments

Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

Elbow method: to choose the best number of clusters (k)



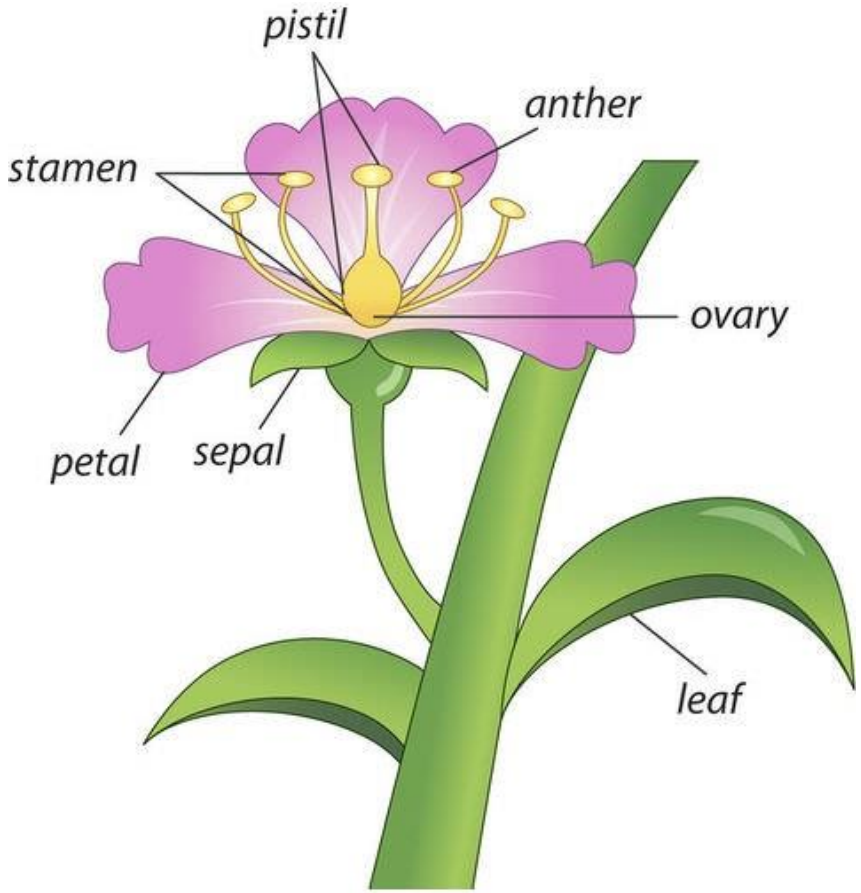
Using a so-called "Elbow" method; It is one of the most popular methods to determine the optimal value of k.

Here, a good number of clusters is 3 at best or 4 if needed.

By considering the Within Cluster Sum of Squares (WCSS), we can select the number of clusters where the change in WCSS begins to level off (called the elbow method).

Source: https://towardsdatascience.com/machine-learning-algorithms-part-9-k-means-example-in-python-f2ad05ed5203

Flowering plant anatomy

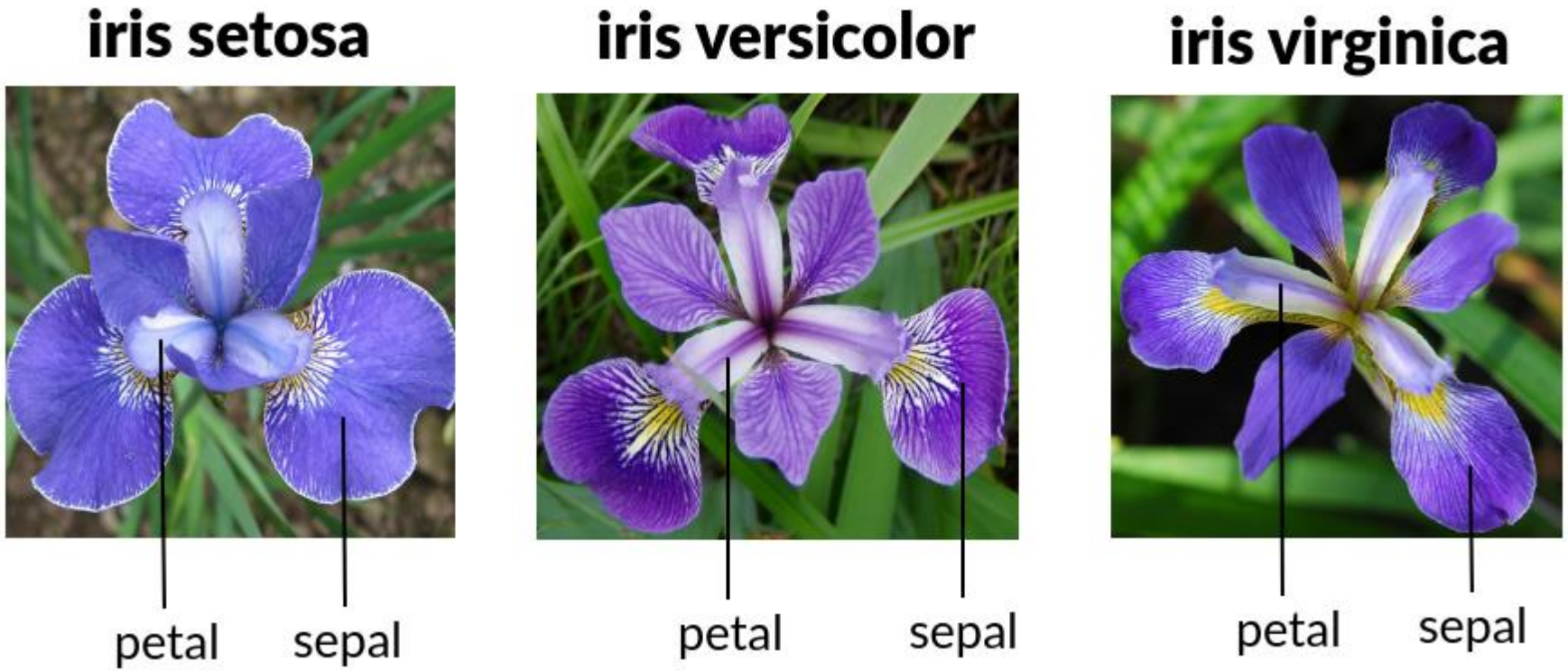


Petal = กลีบดอก

Sepal = กลีบเลี้ยง

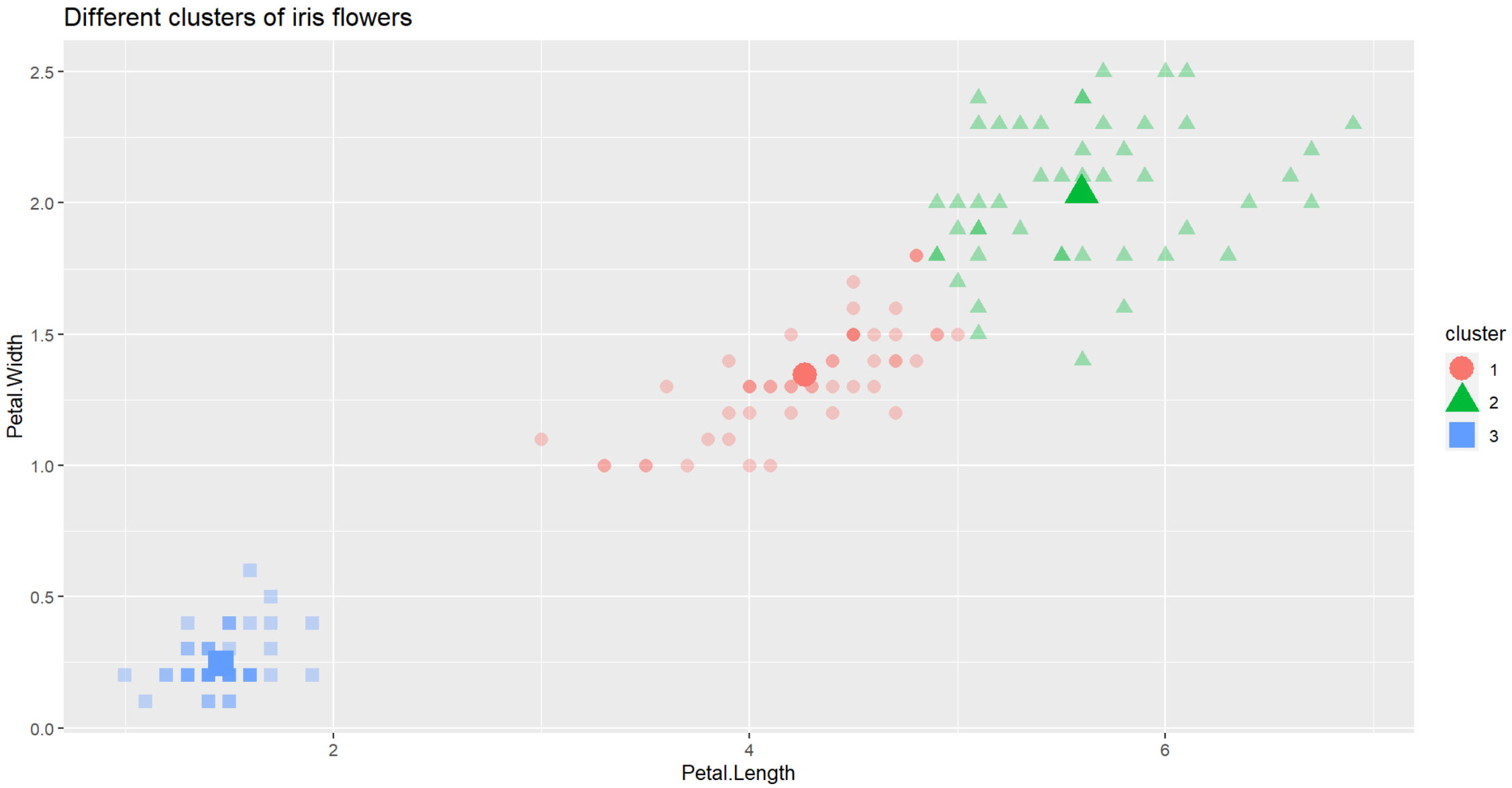
Credit: https://www.ck12.org/book/cbse_biology_book_class_xii/section/2.1/

Dataset: Iris

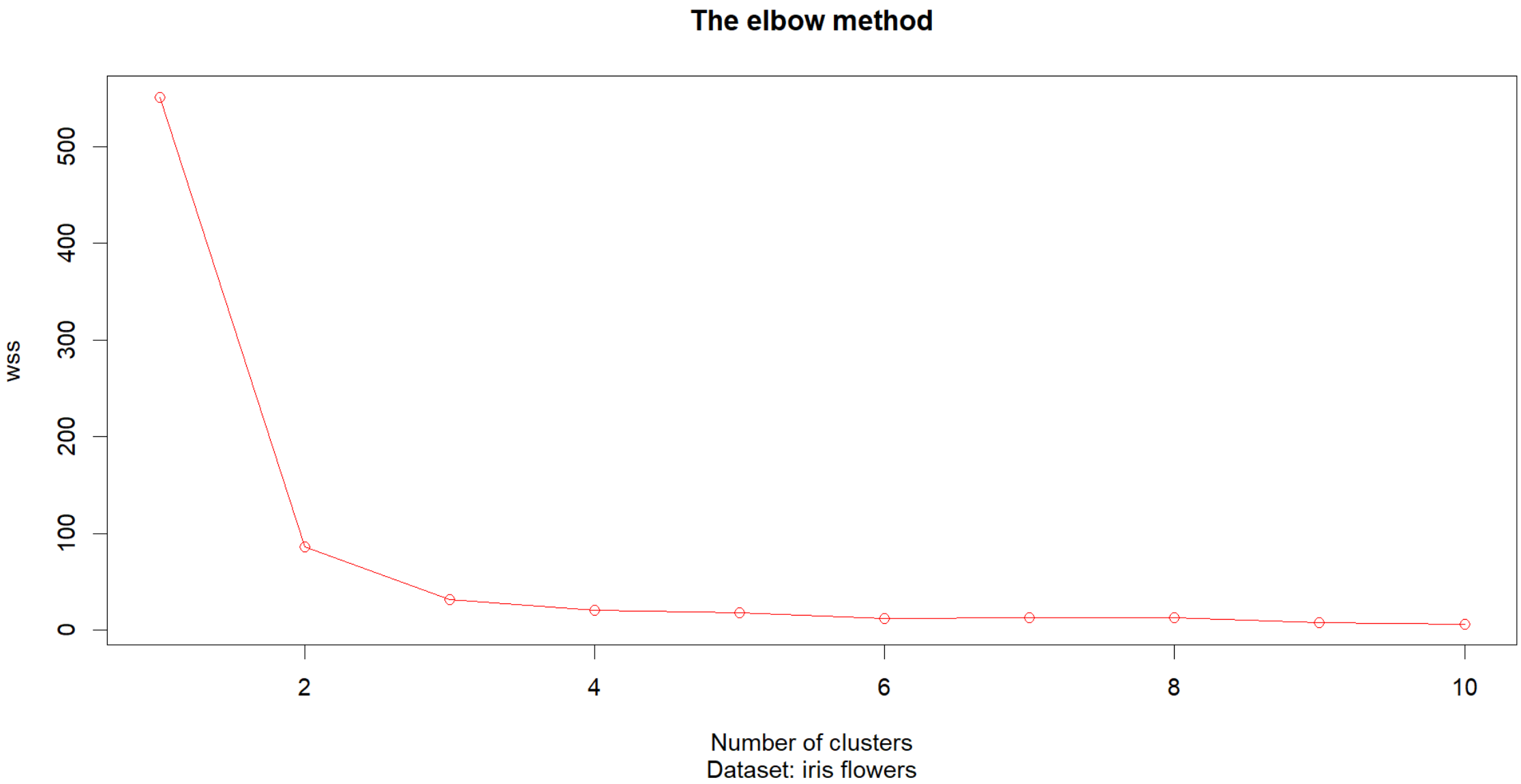


Dataset: https://archive.ics.uci.edu/dataset/53/iris

Clusters of Iris dataset: example from our code



Elbow plot: example from our code



K-prototypes (K-means + K-modes)

NAME	AGE (YEARS)	INCOME (THOUSANDS)	CITY	PREFERRED PRODUCT
Alice	25	30	New York	Shoes
Bob	30	80	Chicago	Electronics
Carol	35	85	Chicago	Electronics
Dave	23	28	New York	Shoes
Eve	40	90	Los Angeles	Electronics

You want to group them into **2 clusters (K = 2)** using:

- **Numerical data**: Age, Income
- **Categorical data**: City, Preferred Product

A simple example: K-prototypes

Each cluster center (the "center" of a group) is called a prototype because it represents the "typical" or most representative point of that cluster.

1. **Pick K = 2** → we want 2 groups.
2. **Place 2 random "centers" (prototypes)** — one for each group.
3. For each customer, calculate how "different" they are from each center using:
   - **Euclidean distance** for numerical data (age, income)
   - **Matching dissimilarity** for categorical data (city, product) → count how many categories are different
   - Combine both into a **total cost**

E.g., cost = $(age_1 - age_2)^2 + (income_1\text{-}income_2)^2$ + (number of mismatches)

A simple example: K-prototypes

4. Assign each person to the **closest prototype**.
5. Update each prototype:
    - For **numerical features**: take the **mean** (like K-Means)
    - For **categorical features**: take the **mode** (most common value, like K-Modes)
6. Repeat until groups stop changing.

A simple example: results

🔷 **Cluster 1: Young, Low-Income, Shoe Lovers in New York**

- Alice (25, $30k, NY, Shoes)
- Dave (23, $28k, NY, Shoes)

Prototype (Center):

- Age: 24 (average)
- Income: $29k
- City: New York (most common)
- Product: Shoes

🔷 **Cluster 2: High-Income, Electronics Buyers**

- Bob (30, $80k, Chicago, Electronics)
- Carol (35, $85k, Chicago, Electronics)
- Eve (40, $90k, LA, Electronics)

Prototype:

- Age: 35
- Income: $85k
- City: Chicago (mode — appears twice)
- Product: Electronics

Try doing it yourself!

Now, any problem?

Because **Income** values are much larger than **Age**, the **Income differences will dominate the distance calculation**.

Example:

- The squared difference in income between Alice and Bob is:
  $(80 - 30)^2 = 2500$
- The squared difference in age is only:
  $(30 - 25)^2 = 25$

Normalization scales values to a similar range (e.g., 0 to 1).

Standardization transforms data to have mean = 0 and standard deviation = 1.

Let's apply **min-max normalization** (scale to 0–1):

$$x_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Solution: normalize or standardize (Z-score) the numerical values

🔷 Normalize Age

- Min = 23, Max = 40 → Range = 17

NAME	AGE	NORMALIZED AGE
Alice	25	(25–23)/17 = 0.118
Bob	30	(30–23)/17 = 0.412
Carol	35	(35–23)/17 = 0.706
Dave	23	(23–23)/17 = 0.000
Eve	40	(40–23)/17 = 1.000

Solution: normalize or standardize the numerical values

🔷 Normalize Income

- Min = 28, Max = 90 → Range = 62

NAME	INCOME	NORMALIZED INCOME
Alice	30	(30–28)/62 = 0.032
Bob	80	(80–28)/62 = 0.839
Carol	85	(85–28)/62 = 0.919
Dave	28	(28–28)/62 = 0.000
Eve	90	(90–28)/62 = 1.000

Now both features are on the **same scale (0 to 1)**, so neither dominates.

Conclusion

Before normalization:

- Distance based on raw numbers → Income dominated

After normalization:

- Age diff: $(0.118 - 0.000)^2 \approx 0.014$
- Income diff: $(0.032 - 0.000)^2 \approx 0.001$
- Now both contribute more fairly

👉 This leads to **more balanced and meaningful clusters**.